OMID.

سایتی که با AI ساختی رو
تا ته قفل کن

سه قفل، دو پرامپت آماده، و همه‌ی اسکیل‌های امنیتی‌ای که الان برای Claude هست؛ برای سایتی که خودت با هوش مصنوعی ساختی.

سایت خودتو بساز · پارت ۳ | ۲ پرامپت آماده | ۶ اسکیل و ابزار امنیتی | ۱۱ صفحه

رایگان

پارت‌های قبلی

هایلایت «سایت» تو پیج @itsomid1.ai

خطرناک‌ترین در، تو کد نیست

سایتی که با AI ساختی و گذاشتی رو گیت‌هاب، کدش رو همه می‌تونن بخونن: حتی رمزی که یه بار توش گذاشتی و بعد پاکش کردی، چون تو تاریخچه‌ی گیت می‌مونه. ولی بزرگ‌ترین خطر یه جای دیگه‌ست: **اکانتت**. پس امنیت سه تا قفل داره:

۱

کد

رمز، کلید و فایل خصوصی تو کد نباشه؛ هیچ اسکریپت ناشناسی اجرا نشه.

۲

حمله

قبل از هکرها، یه هکر هوش مصنوعی سایتت رو واقعاً امتحان کنه.

۳

اکانت

هر کی به اکانت گیت‌هابت برسه، سایتت رو به اسم تو عوض می‌کنه.

۰۲ · اگه فقط ۱۵ دقیقه وقت داری

به این ترتیب، از بیشترین اثر

#	کار	زمان	کجای این راهنما
۱	**تأیید دومرحله‌ای گیت‌هاب**	۲ دقیقه	صفحه‌ی ۱۰ · قفل اکانت
۲	**Push protection** رو روشن کن	۱ دقیقه	صفحه‌ی ۱۰
۳	پرامپت ۱: «مثل هکر بگرد»	۵ دقیقه	صفحه‌ی ۳
۴	پرامپت ۲: «قفلش کن»	۵ دقیقه	صفحه‌ی ۴
۵	/security-review قبل از هر انتشار	۱ دقیقه	صفحه‌ی ۵
۶	Strix، تست حمله‌ی واقعی (پیشرفته)	۲۰ دقیقه	صفحه‌ی ۶

قانون طلایی: هیچ‌وقت رمز، توکن یا کلید API رو تو چت هیچ هوش مصنوعی‌ای تایپ نکن. اگه یه بار کردی، همین امروز عوضش کن.

پرامپت ۱: مثل هکر بگرد

تو Claude Code، داخل پوشه‌ی سایتت، یه **چت تازه** باز کن (چت‌های قدیمی ممکنه رمزت توشون باشه) و این رو عیناً بده:

مثل یه هکر به سایتم نگاه کن. فقط گزارش بده و تا نگفتم چیزی رو عوض نکن.

۱. هر رمز، کلید API، توکن یا فایل خصوصی (مثل .env) تو کد و **کل تاریخچه‌ی گیت**. مقدارش رو چاپ نکن؛ فقط بگو کجاست.

۲. هر جایی که متن بازدیدکننده ممکنه کد بشه (innerHTML، eval، document.write).

۳. هر اسکریپت، فونت یا عکسی که از یه سایت دیگه لود می‌شه.

۴. هر فرمی که اطلاعات رو جایی می‌فرسته.

۵. **لینک‌های** target="_blank" **بدون** rel="noopener".

۶. CSP روی هر صفحه هست؟ چقدر سخت‌گیره؟

۷. هر فایلی تو مخزن که نباید عمومی باشه (یادداشت، بکاپ، اطلاعات مشتری).

آخرش یه جدول: مشکل · خطر (کم/متوسط/زیاد) · راه‌حل.

چرا تاریخچه‌ی گیت؟

پاک کردن یه رمز از کد، فقط نسخه‌ی آخر رو پاک می‌کنه. نسخه‌های قبلی هنوز تو گیت‌هابن و هر کسی می‌تونه بخونه‌شون. برای اطمینان صددرصد، این ابزار متن‌باز کل تاریخچه رو می‌گرده (۳۰ هزار ستاره):

```
gitleaks git -v --log-opts="--all" .
```

نصب: brew install gitleaks (مک)، یا فایل آماده از صفحه‌ی Releases تو github.com/gitleaks/gitleaks (ویندوز). یا فقط به Claude بگو «با gitleaks کل تاریخچه رو بگرد».

اگه رمزی پیدا شد: پاک کردنش کافی نیست. **اول عوضش کن** (رمز جدید، کلید جدید). بعد از کد بردارش. رمزی که یه بار عمومی شده، دیگه رمز نیست.

پرامپت ۲: قفلش کن

وقتی گزارش پرامپت ۱ رو دیدی، تو همون چت اینو بده:

حالا سایت رو قفل کن. سایت ایستاست و رو GitHub Pages می‌ره؛ هدر سرور نداریم، پس همه‌چی با تگ meta.

۱. هر راز پیداشده رو از کد بردار و برای فایل‌های خصوصی .gitignore بساز. اگه رازی تو تاریخچه بوده، بهم بگو عوضش کنم؛ پاک کردن کافی نیست.

۲. رو همه‌ی صفحه‌ها یه CSP سخت‌گیر بذار: default-src 'none' و فقط چیزهایی که خود سایت لازم داره ('self').

اسکریپت و استایل درون‌خطی رو ببر تو فایل که unsafe-inline لازم نشه.

۳. فونت و اسکریپت بیرونی رو بیار تو خود سایت؛ اگه نمی‌شه، integrity (SRI) بذار.

۴. رو همه‌ی صفحه‌ها: meta referrer با strict-origin-when-cross-origin.

۵. همه‌ی لینک‌های تب جدید: rel="noopener noreferrer".

۶. هر جا متن کاربر نشون داده می‌شه: textContent، نه innerHTML.

بعد همه‌ی صفحه‌ها رو باز کن و چک کن چیزی خراب نشده. آخرش لیست تغییرات.

این‌ها یعنی چی، به زبون ساده

CSP	یه نگهبان دم در هر صفحه: فقط کدهای خود سایتت حق اجرا دارن. اگه کسی یه اسکریپت بد تزریق کنه، اجرا نمی‌شه.
SRI	اثر انگشت فایل بیرونی. اگه کسی اون فایل رو تو سرورش عوض کنه، مرورگر لودش نمی‌کنه.
.gitignore	لیست فایل‌هایی که هیچ‌وقت نباید برن رو گیت‌هاب.
textContent	متنی که کاربر می‌نویسه، همیشه متن بمونه و هیچ‌وقت کد نشه.

یه قفل اضافه برای خود Claude: بهش بگو هیچ‌وقت فایل رمزهات رو نخونه. تو فایل .claude/settings.json پوشه‌ی سایتت:

```
{ "permissions": { "deny": ["Read(./.env)", "Read(./.env.*)", "Read(./secrets/**)"] } }
```

اسکیل‌های امنیتی‌ای که الان برای Claude هست

همه رو تو Claude Code نصب می‌کنی. از ساده به پیشرفته:

/security-review

داخلی Claude Code · رایگان با اشتراک

خود Anthropic ساختتش و نصب لازم نداره. قبل از هر انتشار، تغییرهای کدت رو مثل یه متخصص امنیت می‌خونه، خطرها رو با شدت و راه‌حل می‌گه.

بنویس: /security-review

Strix skills

۱۱ هزار ستاره · متن‌باز

اسکیل‌های تیم هکر هوش مصنوعی Strix: پیدا کردن آسیب‌پذیری تو کد، تست نفوذ سایت، تست OWASP Top 10، و رفع مشکل‌ها با اثبات.

```
npx skills add usestrix/strix
```

Trail of Bits

۱٫۸ هزار ستاره · یکی از معتبرترین شرکت‌های امنیت

مهم‌هاش برای سایت: insecure-defaults (تنظیمات پیش‌فرض ناامن)، static-analysis (تحلیل کد با CodeQL و Semgrep)، differential-review (بررسی امنیتی هر تغییر)، supply-chain-risk-auditor (پکیج‌های آلوده).

```
/plugin marketplace add trailofbits/skills
```

Semgrep Guardian

پلاگین رسمی Claude Code

هم‌زمان که Claude کد می‌نویسه، همون کد رو برای آسیب‌پذیری، رمز جاافتاده و پکیج ناامن چک می‌کنه. حساب Semgrep لازم داره (یه بار با مرورگر وارد می‌شی).

```
claude plugin install semgrep@claude-plugins-official
```

gitleaks

ابزار · ۳۵ هزار ستاره

اسکیل نیست، ولی بهترین ابزار برای گشتن رمز تو کل تاریخچه‌ی گیته (صفحه‌ی ۳). Claude خودش اجراش می‌کنه.

Claude Code Security Review

برای حرفه‌ای‌ها

همون /security-review به‌صورت GitHub Action: هر تغییری که رو گیت‌هاب می‌فرستی رو خودکار بررسی می‌کنه و زیرش کامنت می‌ذاره.

github.com/anthropics/claude-code-security-review

Strix: قبل از هکرها، هکت کنه

Strix یه تیم هکر هوش مصنوعیه. متن‌باز و با الا هزار ستاره تو گیت‌هاب. سایتت رو واقعاً امتحان می‌کنه و فقط چیزی رو گزارش می‌ده که تونسته ثابتش کنه. نه یه لیست هشدار بی‌خود.

⚠ **فقط روی سایت خودت.** تست سایت کسی که بهت اجازه‌ی کتبی نداده، غیرقانونیه. خود Strix هم همینو می‌گه.

راه ساده: از داخل Claude Code

۱ **اسکیل‌ها رو نصب کن:** `npx skills add usestrix/strix`

۲ **بگو:** «با Strix یه تست سریع روی سایت خودم بگیر: [آدرس سایتت]». **Claude بقیه رو جلو می‌بره و نتیجه رو با راه‌حل بهت می‌ده.**

راه مستقیم: خط فرمان

```
curl -sSL https://strix.ai/install | bash      # or: pip install strix-agent
strix --target https://your-site.com           # test the live site
strix --target ./your-site-folder              # test the code itself
```

چی لازم داره

- **Docker روشن** (Strix داخلش تو یه محیط ایزوله کار می‌کنه).
- **یه کلید API از یه مدل هوش مصنوعی** (OpenAI، Anthropic، Google...) **یا اشتراک** ChatGPT Plus.
- **بدون نصب هیچی:** نسخه‌ی ابری app.strix.ai **(نیاز به ثبت‌نام).**

از ایران: سایت Docker آی‌پی ایران رو بسته؛ نصب و دانلودش فیلترشکن می‌خواد. خود ایمیج **Strix** رو ghcr.io (رجیستری گیت‌هاب)ه. اگه Docker برات سخته، از قفل ۱ و ۳ شروع کن؛ بیشترین اثر رو اون‌ها دارن.

در اصلی، اکانت گیت‌هابته

هر چقدر کدت محکم باشه، اگه کسی وارد اکانت گیت‌هابت بشه، سایتت رو به اسم تو عوض می‌کنه. این قفل از همه مهم‌تره و از همه سریع‌تر.

۱ · تأیید دومرحله‌ای (۲ دقیقه)

گیت‌هاب ← Settings ← Password and authentication ← Enable two-factor authentication

- با **اپ Authenticator** (مثل Google Authenticator)، نه پیامک.
- **کدهای بازیابی** رو دانلود کن و یه جای امن نگه دار: بدونشون اگه گوشیت گم بشه، اکانت هم رفته.
- اگه گوشی یا لپ‌تاپت پشتیبانی می‌کنه، یه Passkey هم اضافه کن.

۲ · Push protection (۱ دقیقه)

قبل از اینکه یه رمز حتی برسه رو گیت‌هاب، جلوش رو می‌گیره. تو مخزن سایتت: Settings ← Advanced Security ← Push protection ← Enable. **برای مخزن عمومی، اسکن رمز رایگانه.**

۳ · HTTPS اجباری

مخزن ← Settings ← Pages ← **تیک** Enforce HTTPS. **معمولاً روشنه؛ یه بار چک کن.**

۴ · ایمیلت

هر کی به ایمیلت برسه، رمز گیت‌هاب رو هم ریست می‌کنه. تأیید دومرحله‌ای ایمیلت رو هم روشن کن. اگه دامنه‌ی شخصی خریدی، حساب ثبت‌کننده‌ی دامنه هم همین‌طور.

قانونی که هیچ ابزاری جاش رو نمی‌گیره: رمز، توکن و کلید API رو هیچ‌وقت تو چت هوش مصنوعی، دایرکت یا هیچ سایتی که نمی‌شناسی تایپ نکن. Claude برای ساختن سایتت هیچ‌وقت رمزت رو لازم نداره.

ازش اسکرین‌شات بگیر

- [ ] تأیید دومرحله‌ای گیت‌هاب با اپ Authenticator، و کدهای بازیابی یه جای امن
- [ ] تأیید دومرحله‌ای ایمیل
- [ ] Push protection روشن
- [ ] پرامپت ۱ اجرا شد: صفر رمز، صفر کلید تو کد و تاریخچه
- [ ] هر رمزی که یه بار عمومی شده بود، عوض شد
- [ ] پرامپت ۲ اجرا شد: CSP سخت‌گیر رو همه‌ی صفحه‌ها
- [ ] هیچ اسکریپت یا فونت بیرونی بدون SRI
- [ ] .gitignore برای فایل‌های خصوصی، و deny برای خود Claude
- [ ] Enforce HTTPS روشن
- [ ] قبل از هر انتشار: /security-review
- [ ] (پیشرفته) Strix یه بار روی سایت اجرا شد
- [ ] هیچ رمزی تو هیچ چت هوش مصنوعی‌ای

نمره‌ی امنیت سایتت رو ببین

آدرس سایتت رو تو developer.mozilla.org/en-US/observatory بزن. یادت باشه GitHub Pages هدر سفارشی نمی‌ده و بعضی چیزها (مثل frame-ancestors) با تگ meta کار نمی‌کنن؛ پس نمره‌ی کامل برای سایت ایستای گیت‌هاب معمولاً ممکن نیست، و این اشکالی نداره.

همین کارها روی سایت خودم

سایت من (omidkhavari1708-pixel.github.io/itsomid**) رو با Claude ساختم و با پرامپت ۱ همین راهنما بررسیش کردم. نتیجه‌ی واقعی:**

بررسی	نتیجه
CSP (نگهبان دم در)	**روی هر ۱۰ صفحه، سخت‌گیر:** default-src 'none'، **فرم به هیچ جا نمی‌ره** (form-action 'none')
اسکریپت، فونت یا عکس از سایت دیگه	**هیچ.** فونت‌ها کنار خود سایتن.
فرم ورود یا فیلد رمز	**نیست.**
متن بازدیدکننده به‌عنوان کد (eval، innerHTML)	**صفر.**
لینک تب جدید بدون noopener	**صفر.**
رمز، کلید یا توکن تو کد فعلی	**صفر.**
رمز یا کلید تو **تاریخچه‌ی گیت** (۹ کامیت، مخزن عمومی)	**صفر.** فقط یه ایمیل تماس تو یه نسخه‌ی قدیمی بود، که راز نیست.

داستان واقعی: اولش می‌خواستم یه «ورود ادمین» با ایمیل و رمز خودم رو سایت بذارم. رو سایت ایستا یعنی رمز تو کدی که همه می‌تونن بخونن؛ یعنی کلید زیر پادری. نساختمش. سایت ایستا اصلاً ورود لازم نداره.

تو هم همین جدول رو بگیر: پرامپت ۱ (صفحه‌ی ۳) آخرش یه جدول مشکل · خطر · راه‌حل می‌ده. هر خطی که «زیاد» بود، اول همون رو درست کن.

۱۱ تا اشتباه سایت‌هایی که با AI ساخته می‌شن

AI کاری رو می‌کنه که ازش خواستی، نه کاری که امنه. این‌ها رو خودت بپرس:

#	اشتباه	راه درست
۱	**کلید API تو جاوااسکریپت سایت**، چون خواستی سایت مستقیم به یه سرویس وصل بشه	هر کلیدی تو مرورگر، عمومیه. یا سرویسی که کلید نمی‌خواد، یا یه سرور واسط. اگه لو رفته، اول عوضش کن.
۲	**«ورود ادمین» با رمز** روی سایت ایستا	سایت ایستا ورود لازم نداره. برای ویرایش، فایل رو تو گیت‌هاب عوض کن.
۳	**فرم تماس** که اطلاعات رو با یه کلید عمومی جایی می‌فرسته	سرویس فرم معتبر با محدودیت دامنه، یا فقط لینک ایمیل و دایرکت.
۴	**اسکریپت از سایت دیگه** بدون SRI	فایل رو بیار تو خود سایت، یا integrity بذار (پرامپت ۲).
۵	**متن کاربر با innerHTML** نشون داده می‌شه	همیشه textContent. متن کاربر هیچ‌وقت نباید کد بشه.
۶	**فایل خصوصی تو مخزن عمومی:** .env.، بکاپ، یادداشت، اطلاعات مشتری	.gitignore از روز اول، و gitleaks روی کل تاریخچه (صفحه‌ی ۳۳).
۷	**رمز یا توکن تو چت AI**، «که خودش دیپلوی کنه»	Claude برای ساختن سایت رمزت رو لازم نداره. توکن فقط تو فایل یا تنظیمات امن، هیچ‌وقت تو چت.

یه جمله برای آخر هر پرامپت ساخت سایت: «هیچ رمز، کلید یا فایل خصوصی‌ای تو کد یا مخزن نذار؛ اگه لازم شد، اول از من بپرس.»

OMID.

سایتت قفل شد.
حالا تو چی می‌سازی؟

این پارت ۳ از سریال «سایت خودتو بساز»ـه. پارت‌های قبلی، از روز صفر تا طراحی، تو هایلایت «سایت» پیجمه. فالو کن که پارت بعدی رو از دست ندی.

اینستاگرام

instagram.com/itsomid1.ai

منابع

github.com/usestrix/strix · github.com/trailofbits/skills · docs.semgrep.dev · github.com/gitleaks/gitleaks · github.com/anthropics/claude-code-security-review · docs.github.com (2FA, push protection)